Laporan Praktikum Pemrograman Berorientasi Obyek (PBO) – Modul 12

Penyimpanan dan Pembacaan Data ke File (Sistem Kasir Sederhana Berbasis Agile)

Nama : Rafael Rizky Luga Tanjung **Dosen** : Nirwana Haidar Hari, S.Pd., M.Kom.
NRP : 3125522007 **Kampus**: PENS PSDKU Sumenep (2026)

BAGIAN 1 – KONSEPSI AGILE, DATA PERSISTENCE DAN ATOMIC SAFE LOADING

1. Latar Belakang & Analisis Agile Project Kickoff (Sprint P12): Melanjutkan pengembangan proyek Sistem Kasir Sederhana dari Pertemuan 11 dengan kerangka kerja Agile (Scrum). Fokus Sprint P12 adalah Epic: *Persistent Storage, File I/O Resilience & Atomic Safe State Loading*. Sebelumnya pada P11, seluruh data inventaris katalog masih berada secara eksklusif di memori volatil (RAM) yang akan lenyap saat aplikasi ditutup. Pada Sprint P12, arsitektur sistem diekspansi dengan lapisan persistensi berkas (*File Persistence Layer*) menggunakan paket modern java.nio.file. Tanggung jawab baca/tulis dipisahkan secara modular ke dalam class repository (ProdukFileRepository dan TransaksiFileRepository). Sistem kini menjamin ketahanan data terhadap berkas korup melalui teknik *Atomic Safe Load* (verifikasi ke koleksi sementara sebelum swap data).

• **Agile User Stories & Acceptance Criteria:** (1) *Sebagai Kasir*, saya ingin seluruh data katalog produk dan sisa stok tersimpan di disk lokal (produk.txt), sehingga data inventaris tidak hilang saat komputer toko dimatikan. (2) *Sebagai Kasir*, saya ingin aplikasi secara otomatis membaca kembali seluruh objek produk dari disk saat program dibuka ulang. (3) *Sebagai Pemilik Toko*, saya ingin kegagalan membaca file yang rusak tidak menghapus data produk yang sedang aktif di memori (*Fail-Safe State*). (4) *Sebagai Kasir*, saya ingin setiap transaksi yang lunas otomatis tercatat pada riwayat penjualan (transaksi.txt).

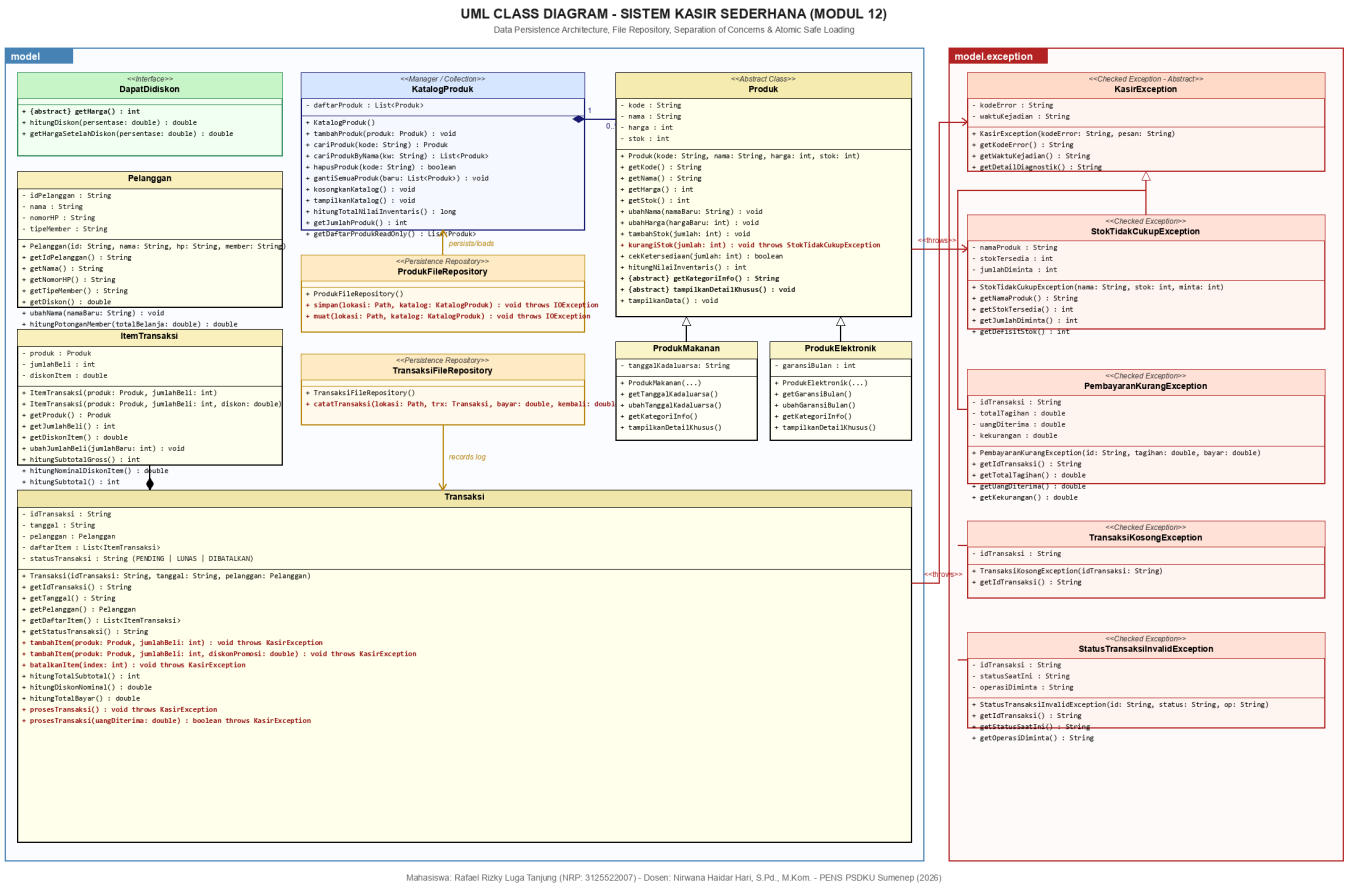
2. Desain Format Berkas & Lapisan Repository Persistensi Data:

Nama Berkas & Lokasi	Format Delimiter Baris	Class Penanggung Jawab	Fungsi & Mekanisme Pengamanan
produk.txt (data/produk.txt)	KODE|NAMA|HARGA|STOK|KATEGORI|EXTRA (6 kolom dipisah pipa)	**ProdukFileRepository**	Menyimpan katalog & merekonstruksi objek ProdukMakanan/ProdukElektronik secara polimorfis. Dilengkapi sanitasi pemisah '|'.
transaksi.txt (data/transaksi.txt)	ID_TRX|TGL|PELANGGAN|MEMBER|TOTAL|BAYAR|KEMBALI|STATUS	**TransaksiFileRepository**	Mencatat log nota transaksi yang lunas secara append (penambahan baris). Berguna untuk rekapitulasi finansial toko.

3. Prinsip Validasi Berkas Eksternal & Atomic Safe Loading: Sesuai mandat Modul 12, data dari file eksternal TIDAK BOLEH dipercaya begitu saja sebelum divalidasi karena dapat diedit manual atau rusak. Pada method muat(), seluruh baris dibaca dan diurai ke dalam koleksi sementara (List<Produk> dataBaru). Pemeriksaan kelengkapan kolom, parsing angka harga/stok, keunikan kode produk, dan tipe kategori dijalankan pada setiap baris. Hanya jika SELURUH baris lolos tanpa error, method katalog.gantiSemuaProduk(dataBaru) dieksekusi. Jika ada 1 baris saja yang cacat, exception seketika dilempar dan katalog aktif di memori tetap 100% utuh tanpa kehilangan data.

BAGIAN 2 – ARSITEKTUR KODE, UML CLASS DIAGRAM DAN TABEL PEMETAAN JAVA

4. UML Class Diagram Terbaru (Modul 12 - Data Persistence Architecture):



Gambar 1. UML Class Diagram Sistem Kasir Sederhana Modul 12 (Pemisahan Tanggung Jawab Repository Persistensi File)

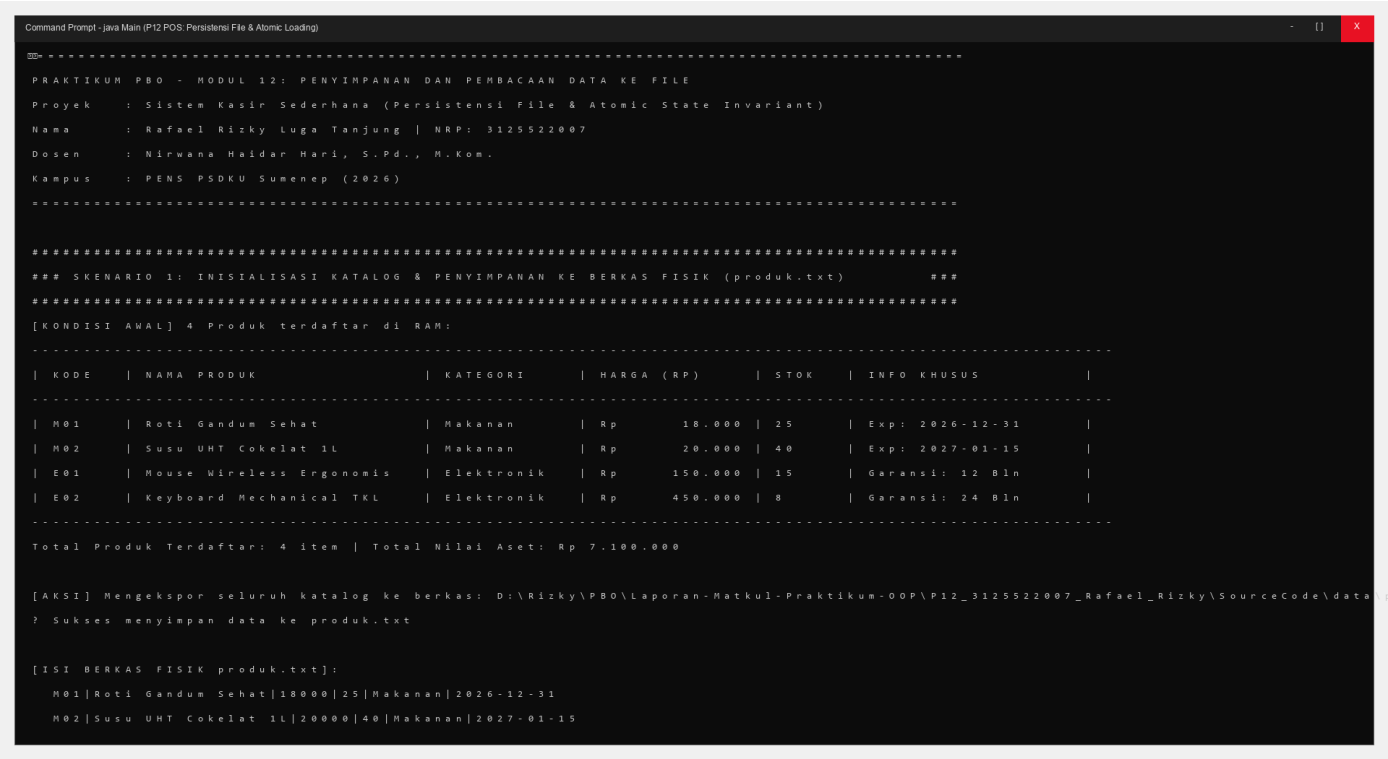
5. Tabel Pemetaan Elemen UML & I/O File ke Source Code Java:

Elemen UML / Komponen	Notasi di Diagram UML	Implementasi pada Source Code Java

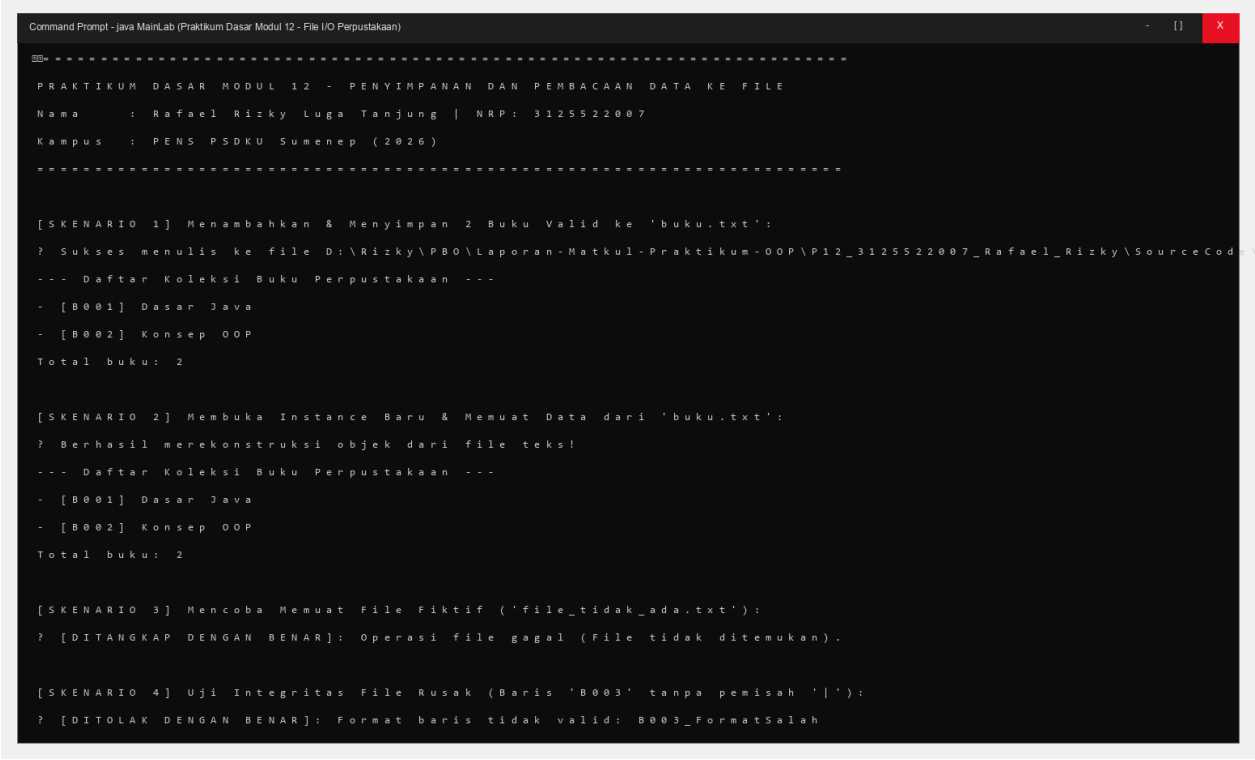
Repository Produk File	class ProdukFileRepository	public class ProdukFileRepository (method simpan & muat dengan Java NIO)
Repository Transaksi File	class TransaksiFileRepository	public class TransaksiFileRepository (pencatatan log transaksi append mode)
Operasi Penulisan Berkas	simpan(lokasi: Path, ...) : void	Files.write(lokasi, barisList, StandardCharsets.UTF_8);
Operasi Pembacaan Berkas	muat(lokasi: Path, ...) : void	List<String> daftarBaris = Files.readAllLines(lokasi, UTF_8);
Atomic Swap Koleksi	gantiSemuaProduk(List<Produk>)	katalog.gantiSemuaProduk(dataBaru); (swap koleksi setelah validasi penuh)
Penanganan Error I/O	<> IOException	throws IOException (menangani file not found, permission, disk failure)

BAGIAN 3 – EKSEKUSI PROGRAM, SKENARIO UJI DAN EVALUASI MODUL 12

6. Bukti Eksekusi Terminal (Main.java & MainLab.java Bebas Error):



Gambar 2. Bukti Running Program Main.java Sistem Kasir Sederhana (8 Skenario Persistensi Teruji 100% Sukses)



Gambar 3. Bukti Eksekusi Praktikum Dasar Modul 12 (MainLab.java: Simpan, Muat & Uji File Rusak Perpustakaan)

7. Ringkasan Hasil Pengujian Komprehensif (Tabel Skenario Pengujian Modul 12):

No	Fokus Skenario Uji	Aksi / Input Pengujian	Hasil yang Diharapkan	Konsistensi State	Status

1	Simpan Produk ke File	Ekspor 4 produk dari memori ke produk.txt	File fisik terbuat dengan format delimiter pipa sah	4 baris di disk	**SUKSES**
2	Muat Sesi Baru (Load)	Buka katalog kosong lalu muat produk.txt	4 objek polimorfis sukses direkonstruksi ke RAM	4 item di RAM	**SUKSES**
3	File Not Found	Muat file fiktif 'produk_fiktif_99.txt'	Catch IOException, lapor berkas tidak ditemukan	Tetap 4 item	**SUKSES**
4	Deteksi Format Rusak	Muat file berisi baris hanya 3 kolom (cacat)	Throw IllegalArgumentException, baris ditolak	Pemuatan batal	**SUKSES**
5	Atomic Safe Loading	Verifikasi status katalog pasca kegagalan uji #4	Katalog memori lama terbukti utuh tidak korup	Tetap 4 item utuh	**SUKSES**
6	Penolakan Kode Duplikat	Muat file dengan 2 baris kode 'M01'	Throw IllegalArgumentException kode ganda ditolak	Tetap 4 item	**SUKSES**
7	Sanitasi Karakter '|'	Simpan nama produk disusupi '|'	Throw IllegalArgumentException sanitasi gagal	File tidak tercemar	**SUKSES**
8	Log Trx & Update Stok	Checkout kasir, simpan log & update stok	transaksi.txt tercatat, sisa stok produk.txt berkurang	Data persisten	**SUKSES**

8. Jawaban Pertanyaan Evaluasi Modul 12:

1. Apa perbedaan data di collection dan data di file?

Data di dalam collection bersifat *volatile in-memory* (berada di RAM komputer) yang cepat diakses selama runtime program, namun akan hilang seketika saat program ditutup atau terjadi gangguan daya listrik. Sedangkan data di dalam file bersifat *persistent non-volatile* (tersimpan di media fisik penyimpanan seperti Harddisk/SSD), sehingga data tetap utuh secara permanen dan dapat dibaca kembali pada sesi eksekusi berikutnya.

2. Mengapa data dari file tetap harus divalidasi?

Karena file eksternal berada di luar kontrol runtime Java dan rentan mengalami modifikasi ilegal dari luar (seperti disunting manual via Notepad, rusak akibat transmisi, atau terjadi duplikasi ID). Validasi wajib diterapkan saat pembacaan untuk memastikan bahwa aturan integritas bisnis domain (seperti kode produk unik, harga positif, stok non-negatif, dan tipe kategori yang sah) tetap terpenuhi sebelum objek direkonstruksi, mencegah pencemaran data (*data corruption*) di dalam sistem.

3. Apa risiko mengosongkan collection sebelum seluruh file selesai diperiksa?

Menimbulkan risiko kehilangan data (*Data Loss*) yang fatal. Jika collection dikosongkan di awal (clear()) lalu terjadi kesalahan format atau kegagalan parsing di baris tengah berkas, eksekusi akan dibatalkan melalui exception. Akibatnya, data lama di memori sudah terlanjur hilang, sementara data baru gagal terbentuk, meninggalkan aplikasi dalam status kosong/rusak. Melalui *Atomic Safe Loading*, seluruh data diuji terlebih dahulu pada koleksi sementara sebelum menimpa memori utama.

4. Mengapa fitur simpan harus dipanggil kembali setelah data diubah?

Karena perubahan data saat program berjalan (seperti pengurangan stok barang saat transaksi kasir atau penambahan produk baru) hanya memutasi objek di memori RAM komputer. Berkas fisik di disk tidak akan berubah secara otomatis. Oleh sebab itu, operasi penyimpanan (simpan()) wajib dipanggil secara berkala atau pasca mutasi untuk menyinkronkan (*flush*) state terbaru dari memori RAM ke dalam berkas fisik penyimpanan.